

Olorin — Engineering Journal

(Archive 1: 2026-07-06 → 2026-07-12)

This is the first archive of the running build log — split off on 2026-07-12 once the live `ENGINEERING_JOURNAL.md` passed ~1000 lines, purely to keep each file readable; nothing here is superseded. Continue reading forward in `ENGINEERING_JOURNAL.md`, which picks up from 2026-07-12 (continued) onward. Full architectural rationale still lives in `OLORIN_PROJECT.md`.

2026-07-06 — Local models + Phase 1 scaffolding

Local models finalized

- Deleted old `olorin`-named `qwen2.5-coder` wrapper (leftover from before the 3-agent split) to avoid naming confusion.
- Rebuilt clean from base weights:
 - `boromir` — `qwen2.5-coder:7b-instruct-q4_K_M`, 16K context via Modelfile
 - `faramir` — `deepseek-r1:8b`, 16K context via Modelfile
- Verified via `ollama list`. `qwen3.5:9b` (unrelated project) left untouched.

Interview angle: explaining *why* a community GGUF or a leftover experimental Modelfile can silently under-perform (low GPU utilization, truncated context) shows you understand Ollama's model-layering

system (named models are Modelfile config layers over shared FROM base weights), not just "I ran ollama pull."

Faramir hallucination test (informal, pre-RAG)

- Asked `deepseek-r1:8b` a pure-recall trivia question (Tolkien lore) with zero retrieval context.
- Result: confidently fabricated plausible-sounding but incorrect details.
- **Conclusion:** this is expected behavior for an 8B parametric model with no grounding — not a config bug. Confirms the RAG-first design is correct: Faramir should never answer from bare memory in the real system, only reason over retrieved context.
- **Design decision (logged in OLORIN_PROJECT.md Section 13):** added `web_search` as a shared V2/V3 tool across all three backends — not a Groq-only feature, since Groq's `llama-3.3-70b-versatile` has the identical blind spot without a tool. Orthogonal to the complexity-scorer routing.
- **Open risk flagged for V2:** `deepseek-r1:8b` emits `<think>...</think>` before real output, which can break tool-call parsing. Must test Faramir's raw tool-calling in isolation before wiring `web_search` into its loop. Boromir (Qwen-based, fine-tuned for function calling) is not expected to have this issue.

Environment setup

- Python **3.11.9** chosen over newer versions — best wheel availability for `chromadb`, `sentence-transformers`, CUDA-enabled `torch` at time of writing.
- `venv` created inside `olorin/` (`python -m venv venv`).
- `.gitignore` written before any other file touched the repo, to guarantee `.env` (API keys) never gets committed.

Phase 1 code — provider abstraction begins

- `config.py` — single source of truth for all env vars, model names, and tunable thresholds (`COMPLEXITY_THRESHOLD`,

`MAX_AGENT_STEPS, CIRCUIT_BREAKER_COOLDOWN_MINUTES)`.
Fails fast with a clear error if `GROQ_API_KEY` is missing, rather than failing silently deep in a provider call later.

- **Interview angle:** "fail fast at config load, not mid-request" is a deliberate reliability pattern, not an accident.
- **providers/base.py** — abstract interface (`BaseProvider`) + normalized `ProviderResponse` dataclass + `ProviderError` (carries `is_rate_limit` flag for the future circuit breaker).
 - **Interview angle:** this is dependency inversion — `LLMClient` and the agent loop depend on this abstract contract, never on a concrete provider's SDK. Swapping Groq → Together AI later is a new class, zero changes to calling code.

providers/groq_provider.py

- Reuses `openai` SDK pointed at `https://api.groq.com/openai/v1` — Groq is OpenAI-compatible, so no bespoke HTTP client needed.
- SDK exceptions (`RateLimitError`, `APIConnectionError`, `APITimeoutError`) caught and normalized into `ProviderError`, with `is_rate_limit` flag set correctly for the future circuit breaker.
- `is_available()` is a cheap pre-flight check (key + client exist) — deliberately does NOT send a real request, to avoid burning rate limit quota on a health check.
- **Verified live:** real response from `llama-3.3-70b-versatile`, 718ms, 46 input / 5 output tokens.

providers/ollama_provider.py

- Same OpenAI-compatible pattern, pointed at `localhost:11434/v1`.
- Takes `model_name` in `__init__` — one instance = one model. `LLMClient` will hold two instances (Boromir, Faramir) rather than this class knowing about personas. Keeps routing logic out of the provider layer.
- `is_available()` calls `models.list()`, NOT a chat

completion — a stray health-check chat call could force an unwanted model swap given only one local model is resident in VRAM at a time.

- **Verified live:** both models responded correctly. Confirmed Ollama's keep-alive/idle-unload behavior firsthand — Boromir loaded, then Faramir loaded and evicted Boromir from VRAM automatically; Faramir stayed resident afterward until manually stopped (`ollama stop <model>`).
- **Note on latency:** first-call latency (~41s Boromir, ~45s Faramir) is dominated by cold-start model loading into VRAM, not inference time. A warm second call to the same model is expected to be dramatically faster. Don't mistake this number for steady-state performance.

Status

- ☒ `config.py`
- ☒ `providers/base.py`
- ☒ `providers/groq_provider.py`
- ☒ `providers/ollama_provider.py`
- ☒ `core/llm_client.py` — routing + circuit breaker verified end-to-end
- ☒ End-to-end test: both backends respond to a hardcoded prompt

`core/llm_client.py`

- Implements `score_complexity()` exactly per Section 7's formula (chunk count, token volume, query length, planning-keyword density).
- **V1 scope decision:** automatic fallback always routes local traffic to Boromir, not a Boromir/Faramir specialist split — that's explicitly a V4 feature (LangGraph refactor) per Section 11. V1's tools are all code-focused, which is Boromir's lane anyway. Faramir stays reachable via explicit `--provider faramir` override for manual testing.
- Circuit breaker: trips only on `RateLimitError` (5 min cooldown, config-urable). Other Groq failures (timeout, connection) fall back to local for that single call without tripping the breaker — a

transient blip shouldn't lock out Groq for 5 minutes.

- **Verified live:** low-complexity query routed to Boromir (9.5s, includes cold VRAM load), high-complexity query correctly routed to Groq (1.8s). Both models gave appropriately cautious "need more context" answers given the test used stubbed/fake retrieved chunks rather than real RAG output — correct behavior, not a bug, since the indexer doesn't exist yet.
- **Still open from Section 7:** query caching (SQLite, sha256 key) and structured logging aren't built yet — bundled in the same V1 checklist item but deferred to keep this commit focused on routing correctness.

Status

- ☒ `config.py`
- ☒ `providers/base.py`
- ☒ `providers/groq_provider.py`
- ☒ `providers/ollama_provider.py`
- ☒ `core/llm_client.py` (routing + circuit breaker)
- ☒ Query cache (SQLite) — verified live
- ☒ Structured logging — verified live
- ☐ Tool registry + agent loop

`memory/cache.py` + `core/logging_config.py`

- Cache key: `sha256(query + context_hash + model_name)`, exactly per Section 7. `context_hash` is itself a hash of the retrieved chunks — meaning cache invalidation on re-index is automatic and free: new retrieval results change the hash, which changes the key, so stale entries are simply never looked up again (not explicitly deleted, just orphaned). No manual invalidation logic needed.
- **Scope decision:** caching only applies when `tools` is `None`. A tool-using response is tied to live state (file contents, search results) at the moment it ran — caching that risks serving stale tool output as if fresh. Tool-free Q&A is safe to cache; tool-driven agent steps are not, in this design.
- Logging: single `get_logger()` factory in `core/`

`logging_config.py`, writes to both console and `logs/olorin.log` (gitignored). Every routing decision — including cache hits, Groq failures, and circuit breaker trips — gets a structured log line with provider, model, latency, and truncated query.

- **Verified live:** first run — real Boromir call (10.4s) and real Groq call (2.0s), both logged correctly with actual latency/provider/model. Second run (identical queries) — both served from cache, 0ms, logged distinctly as `cache HIT`, identical answers returned.

Section 7 of OLORIN_PROJECT.md is now fully implemented and proven: routing, circuit breaker, caching, and logging all working end-to-end. Next: tool registry (Section 9) and the raw ReAct agent loop (Section 9).

`tools/reader.py`, `tools/search.py`, `tools/registry.py`

- `read_file / list_files` fully implemented and scoped to a `repo_root` passed in at dispatch time — never trusted from model-supplied tool-call arguments. Path traversal (`../../../../../etc/passwd`-style) is blocked via `os.path.commonpath` resolution before any file touch.
 - **Interview angle:** same principle as SQL parameterization — never let untrusted input (here, LLM-generated tool arguments) control the security-relevant part of an operation. Directly relevant if a prompt-injected instruction hidden in an indexed file tries to redirect a tool call.
- `search_codebase` is a deliberate stub: fails LOUDLY with an explicit "not implemented, indexer not built yet" message rather than a silent empty result — lets the agent reason around the gap (fall back to `list_files/read_file`) instead of assuming something broke silently.
- `tools/registry.py` holds both the OpenAI-format tool schemas AND the dispatch function (`execute_tool`) in one file — adding a new tool later (e.g. `web_search` in V2/V3) is a two-part change entirely within this module: schema + dispatch branch, nothing else touched.
- **Verified live:** `list_files` found 14 real repo files, `read_file`

correctly read `config.py` (3365 chars), the search stub returned its intended error message, an unknown tool name returned a graceful error dict instead of raising, and the path-traversal attempt was correctly blocked.

Status

- `config.py`
- `providers/base.py`
- `providers/groq_provider.py`
- `providers/ollama_provider.py`
- `core/llm_client.py` (routing + circuit breaker + cache + logging)
- `tools/reader.py`, `tools/search.py` (stub), `tools/registry.py`
- `core/agent.py` — raw ReAct loop, working end-to-end

`core/agent.py` — raw ReAct loop

- Built exactly per Section 9's pseudocode: send messages + tools → if `finish_reason == "stop"` return final answer → if `"tool_calls"` execute each via `tools/registry.execute_tool`, append results as `role: "tool"` messages, loop. Hard-capped at `config.MAX_AGENT_STEPS`.
- `build_system_prompt()` is deliberately the plain V1 version — OLLORIN.md per-repo injection (Section 8) is explicitly V2, not built yet.

Real bug found + fixed: Ollama tool-calling transport quirk

- **First run failed silently-ish:** asked the agent to read `config.py`. Boromir correctly reasoned it needed `read_file`, but the loop returned immediately with a "final answer" that was literally the raw JSON tool call as text: `{"name": "read_file", "arguments": {"path": "config.py"}}`. `finish_reason` came back `"stop"`, not `"tool_calls"` — the

structured tool-call mechanism never fired.

- **Diagnosed via web search, not guessed:** this is a well-documented, ecosystem-wide issue with Ollama's OpenAI-compatible endpoint (`/v1/chat/completions`) and qwen2.5-coder specifically — confirmed independently in GitHub issues from Cline, OpenCode, and a Hermes-based agent project, all hitting the identical symptom. Ollama's own docs mark the `/v1` endpoint as experimental. Not a bug in our code, not a Boromir reasoning failure — a transport-layer gap in the OpenAI-compat shim specifically.
- **Root-cause fix (not just a workaround):** rewrote `OllamaProvider` to use the official `ollama` Python library against Ollama's NATIVE `/api/chat` endpoint, which is documented to handle tool calling correctly, instead of the experimental OpenAI-compat shim. This broke symmetry with `groq_provider.py` (which correctly still uses `openai`, since Groq's compat layer is stable) — acceptable, since `BaseProvider` means nothing outside this file cares which client library is used internally.
- **Second issue surfaced by the switch, fixed the same day:** Ollama's native API expects a genuinely different message shape than OpenAI's convention for the two message types involved in tool use — `tool_calls[].function.arguments` must be a real dict (not a JSON string), and tool-result messages need a `tool_name` field instead of OpenAI's `tool_call_id`. `core/agent.py` builds message history in OpenAI-canonical shape (correct, since that's what Groq needs unmodified) — so `OllamaProvider` now translates bidirectionally: `_try_parse_fallback_tool_call()` / output normalization for native→OpenAI-shape (going out to the agent), and the new `_translate_messages_for_native()` for OpenAI→native-shape (going in to Ollama). Both translations live entirely inside `providers/ollama_provider.py` — `core/agent.py` never needed to change.
- **Kept the JSON-in-content fallback parser** as defense-in-depth even after the native-API fix — confirmed inert (never fires) on a correctly- behaving response, so it costs nothing to keep and protects against any future model/template combination that still misbehaves.
 - **Interview angle:** this whole arc — symptom → root-cause

diagnosis via evidence, not guesswork → fix at the correct architectural layer → a second latent issue surfaced immediately by the first fix → fixed the same way — is the strongest engineering story in the project so far. It also concretely demonstrates why `BaseProvider` exists: two rounds of provider-specific quirks were absorbed entirely inside one file, with zero changes required to `core/agent.py`, `core/llm_client.py`, or `tools/registry.py`.

- **Verified live, final clean run:** step 1 — native `tool_calls` fired correctly (no fallback needed), `read_file` executed for real. Step 2 — model reasoned over the actual file content and produced a fully grounded, accurate answer citing real variable names, real threshold semantics, and the actual sanity-check behavior from the file — not a generic guess. Latency dropped to 2.9s/8.4s (warm model) from the 40s+ cold-start numbers seen earlier.

Status

- ☒ `config.py`
 - ☒ `providers/base.py`
 - ☒ `providers/groq_provider.py`
 - ☒ `providers/ollama_provider.py` (native API + bidirectional message translation + fallback parser)
 - ☒ `core/llm_client.py` (routing + circuit breaker + cache + logging)
 - ☒ `tools/reader.py`, `tools/search.py` (stub), `tools/registry.py`
 - ☒ `core/agent.py` — verified working end-to-end, fully clean run
 - ☐ Indexer (Rust file walker, tree-sitter, GPU embeddings, ChromaDB) — next
 - ☐ Basic CLI (`cli.py`)
-

2026-07-08 — Indexer: Rust file walker (`indexer_core/`)

Rust toolchain

- Rust was not previously installed on this machine. Installed via `rustup` (winget), verified with `rustc --version / cargo --version` before writing any code.

indexer_core/ scaffolded

- `Cargo.toml`: `ignore` + `rayon` (parallel, `.gitignore`-aware walking — same crate `ripgrep` uses), `serde/serde_json` (JSON output), `sha2` + `hex` (file hashing).
- `src/main.rs`: `WalkBuilder::build_parallel()` walks the repo; hashing happens *inside* the parallel visitor closure, so both traversal and SHA-256 hashing are parallelized, not just the directory walk. Streams each file through the hasher in 8KB chunks rather than loading whole files into memory. Extension → language-label mapping for the future tree-sitter chunker. Unreadable files are skipped, not fatal — one bad file doesn't kill the whole walk. Output: single JSON array on stdout matching the Section 8 contract exactly (`path`, `size`, `language`, `hash`, `modified`).
- Added `indexer_core/target/` and `logs/` to `.gitignore` (`target/` for Cargo build artifacts; `logs/` was referenced as "gitignored" in an earlier entry but had never actually been added to the file).

Real bug #1: .gitignore silently ignored — require_git default

- **First run:** `indexer_core.exe ..` dumped hundreds of files from `venv\Lib\site-packages` straight to stdout — exactly the directory `.gitignore` excludes.
- **Diagnosed, not guessed:** the `ignore` crate's `WalkBuilder` only applies `.gitignore/.git/info/exclude/global-gitignore` rules when the walked path is inside an actual git repository (`require_git` defaults to `true`). `git init` had never been run

in `olorin/`, so every `.gitignore` rule was silently inert — the walker fell back to "ignore nothing," full stop.

- **Fix:** added `.require_git(false)` to the `WalkBuilder` call in `main.rs`, with an inline comment explaining why — Olorin's ignore scope is defined by the `.gitignore` file as config, not by git repo status, so this shouldn't depend on whether `git init` has been run.

Real bug #2: `.gitignore` written with literal quote characters

- **Second run, same symptom:** after the `require_git(false)` fix and a rebuild, `venv/` was *still* being walked in full. Identical symptom to bug #1, different root cause — systematic re-diagnosis required rather than assuming the first fix was incomplete.
- **Root cause:** `.gitignore`'s actual on-disk content was:

```
".env"  
"__pycache__/"  
"*.*pyc"  
"venv/"  
...
```

Every pattern line had literal double-quote characters wrapped around it — almost certainly from an earlier PowerShell command that quoted each pattern for the shell, but wrote the quotes into the file itself instead of stripping them. Git's `.gitignore` syntax does not treat leading/trailing `"` as delimiters; the actual pattern being matched was the literal string `"venv/"` (quotes included), which matches no real path, so every rule in the file was silently a no-op.

- **Fix:** rewrote `.gitignore` with the same patterns, quote characters removed.
- **Interview angle:** two bugs, identical external symptom (`venv/` fully walked), completely different root causes — one a crate default behavior, one a malformed config file. Good illustration of why you verify the actual fix resolved the actual symptom instead

of assuming the first plausible-sounding cause was the only one.

Verified live, clean run

- Final `indexer_core.exe` .. output: 20 real project files (`config.py`, `core/`, `providers/`, `tools/`, `memory/`, `indexer_core/{Cargo.toml, Cargo.lock, src/main.rs}`, both markdown docs). Zero `venv/`, `__pycache__/`, `logs/`, `*.db`, or `indexer_core/target/` entries — every exclusion category confirmed working, not just spot-checked.
- **Benchmarked** (`Measure-Command`): **301ms** for the full walk — comfortably under the sub-1-second target claimed in `OLORIN_PROJECT.md` Section 13. This is a real number on a modest-sized repo (~20 tracked files plus the full `venv/` tree being traversed and filtered), not a guess.

Status

- ☒ Rust file walker (`indexer_core/`) — parallel walk + parallel hashing, `.gitignore`-aware, JSON output matching Section 8 contract, benchmarked at 301ms
- ☒ tree-sitter chunker (`indexer/chunker.py`) — verified live against `core/agent.py`: correctly split into function/class chunks with accurate line ranges, deliberate class/method overlap confirmed working as designed
- ☒ GPU embeddings (`indexer/embedder.py`) — verified live: Device: `cuda` confirmed (not a silent CPU fallback), 384-dim output matches `bge-small-en-v1.5`'s known dimensionality, query embed at 20ms once warm. BGE asymmetric query/passage instruction split implemented as two separate functions (`embed_chunks` / `embed_query`) rather than a flag, so the two can't be accidentally swapped.
- ☒ ChromaDB store (`indexer/store.py`) — verified live, full pipeline end to end: chunked + embedded + stored `core/agent.py` for real, then queried "how does the agent loop handle tool calls" and got back `Agent` (distance 0.51) and `run` (distance 0.53) ranked above the less-relevant `build_system_prompt` (0.81) — correct semantic ranking, not just "query didn't error."

Chroma data deliberately stored outside any indexed repo, at a single global `~/.olorin/chroma_db`, with one collection per repo keyed by a hash of its resolved path — indexing someone else's repo never writes into their working tree.

- `□` Swap `search_codebase` stub in `tools/search.py` for the real `indexer/store.py` query — next
- `□` Basic CLI (`cli.py`) — needs an `index <path>` command that ties `walker → chunker → embedder → store` together as one pipeline (including the hash-based skip-unchanged-files logic that `get_indexed_file_hash()` was built to support)

Indexer (Section 8) is now fully built and individually verified at every stage. What's left is wiring, not new subsystems: a `cli.py` index command to orchestrate `walker→chunker→embedder→store` as one pass over a repo, and pointing `tools/search.py`'s `search_codebase` at `indexer/store.py`'s `query()` instead of its stub error message.

2026-07-09 — V1 complete: **search_codebase** live, **cli.py** built, two real bugs found and fixed

tools/search.py — swapped the stub for a real ChromaDB call

- Thin wrapper: `get_collection(repo_root) → collection.count() == 0` check (repo not indexed yet — explicit, honest error, same philosophy as the old stub) → `indexer/store.py`'s `query()` → reshaped into `{chunks: [...]}`.
- `tools/registry.py`'s `execute_tool()` now forwards `repo_root` into `search_codebase`, matching the same injection pattern already used for `read_file/list_files` — never trusted from model-supplied arguments.

- Updated the tool schema description and `core/agent.py`'s system prompt, both of which still described the tool as unimplemented.

`cli.py` — the actual V1 entry point, built

- `index <path>: walker (subprocess) → per-file hash comparison against get_indexed_file_hash() (skip unchanged, delete_file() + full re-chunk on changed) → chunk_file() → embed_chunks() → upsert_chunks(). Reports skipped/reindexed/total-chunks + timing.`
- `ask "<query>" [--path] [--provider] [--force-local]: thin wrapper around Agent.run().`
- Deferred the `torch/chromadb/sentence-transformers` imports to inside `index()` only, so `ask` and `--help` don't pay that import cost.

Real bug found + fixed: absolute paths in indexer output

- **Symptom:** first `python cli.py index .` run indexed real files, but every chunk's `file` metadata was a full absolute path (`C:\Users\Aryavart\olorin\tools\__init__.py`) instead of the Section-8-contract relative path (`tools/__init__.py`).
- **Root cause:** `cli.py` calls `os.path.abspath(path)` before invoking the Rust walker, and `main.rs` had never actually enforced the relative-path half of its own documented JSON contract — it just did `path.to_string_lossy()` verbatim. This had gone unnoticed because the earlier walker-only smoke test happened to pass a relative arg (`..`).
- **Fix, at the correct layer (Rust, not a Python-side path-stripping patch):** canonicalize both `repo_path` and each walked file path, `strip_prefix()` the repo root off, forward-slash-normalize for cross-platform stability. First compile attempt failed (E0507: can't move a captured `PathBuf` into an `FnMut` closure that's invoked once per worker thread) — fixed by wrapping `repo_root_canon` in an `Arc` and cloning it per-thread, same pattern already used for the shared `results` vec.
- **Why this matters beyond cosmetics:** absolute paths would

have silently broken `get_indexed_file_hash()`'s file-identity comparisons across machines/checkouts, and skewed `tools/reader.py`'s `read_file` semantics (it happened to still work by coincidence — `os.path.join` treats an absolute second argument as the whole path — but that's not a design anyone should rely on).

Real bug found + fixed: fenced-JSON tool-call not recognized

- **Symptom:** live `ask` run against a real indexed repo — Boromir needed `search_codebase`, but the loop returned a "final answer" that was literally a ````json ... ```` fenced block containing the tool-call JSON. `finish_reason` came back "stop".
- **Diagnosis:** the same underlying transport quirk logged 2026-07-06 (model emits tool-call JSON into plain content instead of the structured field), but a new shape of it — the existing `_try_parse_fallback_tool_call()` guard only matched bare `{...}`, so a markdown-fenced payload slipped past it silently.
- **Fix:** `_try_parse_fallback_tool_call()` now strips a leading/trailing ````` or ````json` fence before the existing brace-matching check runs. Fixed entirely inside `providers/ollama_provider.py` — no changes to `core/agent.py` or `tools/registry.py`, consistent with the dependency-inversion principle already established for this exact class of bug.
- **Interview angle:** a good "root cause wasn't fully closed the first time" story — the original fix (native API + bidirectional translation) was correct and necessary, but the defense-in-depth fallback parser it left behind had a narrower net than the actual space of ways a small model can misformat structured output. A defensive fallback is only as good as the shapes it was tested against.

Verified live, clean end-to-end run — V1 demo target achieved

- `python cli.py index .: 25 files walked (0.14s), 74 chunks embedded and stored, 10.23s total, correct relative paths in every chunk's metadata.`
- `python cli.py ask "how does the agent loop handle tool calls?": step 1 — Boromir correctly called search_codebase with real arguments; step 2 — reasoned over real retrieved chunks from core/agent.py and gave a fully grounded 8-point answer.`
- **Fact-checked the answer against the actual source line by line** — every claim (message construction, `MAX_AGENT_STEPS` loop, exact `llm.chat()` argument names, `finish_reason` branching, tool-call message ordering, `execute_tool()` call shape, tool-result message shape, unrecognized-`finish_reason` fallback) matched the real code. No hallucinated details.
- **This is the V1 demo target from Section 16, achieved for real** (the specific query differs from the original "what does the auth module do?" example since this repo has no auth module — an equivalent grounded query against real indexed code was used instead).

Status — V1 is complete

- ☒ `config.py`
- ☒ `providers/base.py`
- ☒ `providers/groq_provider.py`
- ☒ `providers/ollama_provider.py` (native API + bidirectional translation
 - fallback parser, now fence-aware)
- ☒ `core/llm_client.py` (routing + circuit breaker + cache + logging)
- ☒ `tools/reader.py`, `tools/search.py` (live), `tools/registry.py`
- ☒ `core/agent.py`
- ☒ Full indexer pipeline (`indexer_core/`, `indexer/chunker.py`, `indexer/embedder.py`, `indexer/store.py`)
- ☒ `cli.py` (index + ask)
- ☒ **V1 demo target verified live and fact-checked**

Five real bugs found and fixed at the correct architectural layer across this project so far (full stories in Section 13 and this journal): the two-part Ollama transport/schema quirk, the `require_git` default, the quoted-`.gitignore` bug, the absolute-path walker bug, and the fenced-JSON fallback-parser gap.

Next direction (per Aryavart, 2026-07-09): reduce cold-start/reload overhead

- Every `python cli.py` invocation currently reloads `bge-small-en-v1.5` from disk into VRAM from scratch (~1-2s), even for a single query embed — because `cli.py` is a fresh process every call, `indexer/embedder.py`'s module-level `_model` cache buys nothing across invocations, only within one.
 - Flagged as the next priority, ahead of V2 memory features: investigate a persistent local process (daemon-style — related to V4's system-tray plan but pulled earlier and scoped narrower: just "keep the embedding model warm across CLI calls," not the full tray/hotkey/watcher feature set) so `ask` doesn't pay a reload tax on every call.
 - Not yet designed — next session's focus.
-

2026-07-09 (continued) — V1.5 profiling instrumentation, two profiling bugs, Boromir model swap

`core/profiling.py` built — the 9-stage instrumentation plan from earlier today

- Shared `span()` context manager + `record()` for pre-measured durations, wired into `cli.py` (`agent_import_chain`, `total_request`), `indexer/embedder.py` (`embedder_init`, `query_embedding`), `indexer/store.py` (`chroma_init`,

`vector_search`), and `providers/ollama_provider.py` (Ollama's own server-reported `load_duration/total_duration`, more accurate than wall-clock guessing for splitting model-load from real inference).

- `cli.py ask --profile` prints the full breakdown after the answer.

Real bug found + fixed: profiling report double/triple/quadruple-counted nested spans

- **Symptom:** first live `--profile` run reported `sum of recorded spans` far exceeding the actual wall clock (117350ms summed vs. 58024ms wall clock), producing a nonsensical *negative* "unaccounted" number.
- **Root cause:** `report()` naively summed every span, but most spans are nested, not sequential — `total_request` already contains `chroma_init/embedder_init/query_embedding/vector_search`/every Ollama call, and each Ollama call was independently recorded four overlapping ways (`total`, `model_load`, `inference`, `wall_clock`). Summing all of that flat quadruple- and quintuple-counted huge chunks of time.
- **Fix:** added a `top_level: bool` flag to `span()/record()`. Only `agent_import_chain` and `total_request` (the two genuinely sequential, non-overlapping phases in `cli.py`'s `ask()`) are marked top-level and summed for the accounting total; everything else still prints for the detailed nested breakdown but is excluded from the sum. Ollama instrumentation rewritten to record a true partition — `model_load + inference + client_overhead` now sum *exactly* to wall-clock latency, no overlap.
- **Verified:** re-run after the fix produced `unaccounted` values of ~78-95ms across three runs — small, sane, and consistent, versus the original negative number.

Real finding: HF Hub warning was a genuine, fixable cost, not noise

- `sentence-transformers` pings Hugging Face Hub to check for a newer model revision on every load, even with `bge-small-en-v1.5` fully cached locally — a live network round-trip sitting inside `embedder_init`, correlating exactly with a live 5847.9ms measurement (vs. the "~1-2s" the code's own comments expected).
- **Fix:** `os.environ.setdefault("HF_HUB_OFFLINE", "1") / TRANSFORMERS_OFFLINE` set in `indexer/embedder.py`, before the `sentence_transformers` import (some of its config is read at import time). Real fix, not a workaround — the model's already downloaded, so the network check was never buying anything.
- **Verified:** `embedder_init` dropped from 5847.9ms → 983.2ms (83% cut) on the next run, no more HF warning in the logs.

What the profiling data actually revealed: import cost, not reload cost, dominates

- `agent_import_chain` (the cascade: `core.agent` → `core.llm_client` → `providers` → `tools.registry` → `tools.search` → `indexer.store/indexer.embedder`, i.e. `torch/chromadb/ sentence-transformers`) measured 8546.5ms and 6807.1ms across two consecutive runs — 66%+ of total wall clock on a single-step query with no tool call at all. `embedder_init/ chroma_init` (now that the HF fix is in) are comparatively small once the module-level singleton caching kicks in.
- **This directly answers the original V1.5 decision gate (OLORIN_PROJECT.md Section 11):** the module-level `embedder/Chroma` caching already does its job *within* one process — the dominant cost is Python import overhead paid fresh on **every process launch**, not reload-within-a-session. This is a real, measured signal (not an assumption) that a persistent process is justified. Daemon design is next session's concrete next step, no longer speculative.

Real bug found (Boromir, not this instrumentation): a third tool-call malformation shape

- Same live profiling session surfaced Boromir (still qwen2.5-coder at this point) producing two more bad outputs on the *identical* repeated query: one flat non-answer with no tool-call attempt at all, and one prose-prefixed bare-JSON tool call (a sentence of reasoning followed by unfenced JSON) — a third distinct shape after bare-JSON (2026-07-06) and markdown-fenced JSON (earlier today), each of which required its own special case in `_try_parse_fallback_tool_call()`.
- **Assessment logged, not yet fixed via parser patching:** three shapes in one session is a pattern, not noise — chasing each new malformation individually doesn't guarantee the fourth is caught. This assessment directly motivated the model swap below instead of a fourth parser special case.

Decision: Boromir's base swapped from qwen2.5-coder to qwen3:8b

- Verified via current web search (not assumed from training data, which would be stale for a model landscape moving this fast) that Qwen3 has native tool-calling support built into its chat template, and Qwen3 8B is repeatedly named the most reliable small local tool-calling model across independent 2026 benchmarks — a structural difference from qwen2.5-coder, which serializes tool descriptions into the prompt and the model decides for itself whether to emit structured output.
- Same VRAM class as the model it replaces (~5.2GB vs. 4.7GB, both comfortable in the 8GB budget), Apache 2.0 licensed, with a genuine hybrid thinking/non-thinking toggle.
- **Trade-off stated explicitly:** qwen3:8b is general-purpose, not code-specialized like qwen2.5-coder was. Accepted because Boromir's actual V1 job — calling `read_file/list_files/search_codebase` and reasoning over retrieved code — is comprehension and tool orchestration, not code generation, which is exactly the profile tool-call reliability should be weighted for.
- **Alternative considered and set aside:** Llama-3-Groq-8B-Tool-Use scored higher on raw tool-calling benchmarks (89% BFCL) and has a fun narrative tie-in given Olorin already uses Groq, but it's an older 2024 specialist fine-tune with unconfirmed active

maintenance in the Ollama library today. Qwen3:8b is the more current, broadly-supported pick.

Discovered post-swap: Faramir and Boromir now share a base architecture

- `ollama show` on both models reports `architecture: qwen3` — not a bug. DeepSeek's `deepseek-r1:8b` distill (Faramir) is R1's reasoning traces trained onto a Qwen3 base, so the shared label reflects real ancestry.
- Confirmed genuinely distinct via `ollama show`: different license (MIT/DeepSeek vs. Apache 2.0/Qwen), different chat-template stop tokens (DeepSeek/R1 convention vs. ChatML — prompt formatting isn't interchangeable despite the shared architecture label), different reported max context (131072 vs. 40960).
- **Real bug found and fixed as a direct consequence:**
OllamaProvider had no `think` control at all — both personas were constructed identically. Now that Boromir has a genuine thinking mode (`qwen2.5-coder` never did), it would have silently inherited Ollama/Qwen3's default thinking behavior, working against Boromir's whole design intent (fast, task-focused, no deliberation). Added `think: bool | None` to `OllamaProvider.__init__()`; `core/llm_client.py` now constructs Boromir with `think=False` and Faramir with `think=True` — explicit per-persona control instead of a shared default neither model asked for.

Status

- ☒ `core/profiling.py` — 9-stage instrumentation, two real profiling bugs found and fixed, verified live
- ☒ `HF_HUB_OFFLINE` fix — verified live, 83% cut to `embedder_init`
- ☒ V1.5 decision gate reached: import overhead, not reload overhead, is the dominant cost — daemon design is justified and is next session's concrete task
- ☒ Boromir base swapped to `qwen3:8b`, `think` control added and

wired per-persona

- ☐ Daemon architecture design — still next session's task
 - ☒ Verified: 3/3 consecutive `ask` runs post-swap produced clean, structured `tool_calls` (visible directly in the agent log, no fallback-parser trigger, no malformed JSON in any shape). All three final answers fact-checked against `core/agent.py`'s real source — accurate, including a system-prompt detail ("includes the repository root") that was specifically re-verified against `build_system_prompt()`.
-

2026-07-10 — think override extension point, think:false confirmed honored, daemon deliberately deferred

Extension point: per-call `think_override` threaded through the whole stack

- Motivation: keep the possibility open of putting Boromir into thinking mode for a specific heavy task in a later version, without having to restructure the provider/routing layers when that need actually shows up. Added `think_override: bool | None = None` to `BaseProvider.chat()`'s abstract signature, `GroqProvider.chat()` (accepted, ignored — Groq has no thinking-mode concept), `OllamaProvider.chat()` (real effect: falls back to the instance's standing `think` default when not given), and `LLMClient.chat()/_route()` (passed straight through to whichever provider ends up handling the request). Nothing in V1 sets this yet — it's a deliberate no-op extension point, not a feature.

Diagnostic added: directly confirm whether `think` is honored

- Added a log line in `OllamaProvider.chat()` reading Ollama's own `message.thinking` field and reporting `think_requested=<bool>` `thinking_produced=<yes/no>` `thinking_chars=<n>` on every call — replaces inferring thinking behavior indirectly from latency with a direct, per-call confirmation.
- **Verified live:** every one of Boromir's calls across four full `ask` runs logged `think_requested=False` `thinking_produced=no` `thinking_chars=0` — `think:false` is genuinely honored, not silently ignored.

Re-measured with realistic tool-using queries — findings that complicated, not just confirmed, the daemon case

- On short single-step queries (2026-07-09's data), `agent_import_chain` was 66%+ of total wall clock — seemingly a clear daemon justification.
- On longer, more realistic tool-using queries (this session), `total_request` grew substantially because step 2's `ollama_inference` ballooned to 13-27 seconds on multi-paragraph structured answers — real generation time, not overhead. Against that, `agent_import_chain`'s share of total wall clock dropped to 17-30%. **A daemon eliminates import overhead but cannot touch inference time, which is the actual largest cost on realistic queries.**
- Side observation, flagged as a hypothesis not a confirmed fact: `ollama_client_overhead` was consistently ~2050ms on every step-1 call and ~3-6ms on every step-2 call, across all four runs — possibly a one-time per-session cost (grammar/template compilation for constrained/tool-schema output) that a daemon would only pay once per model load instead of once per process. Not chased further this session.

Decision: daemon justified by the numbers, deliberately DEFERRED

- Discussed directly with Aryavart, who raised the right question: is

it wise to build persistent-process infrastructure now, before V1 has even finished evolving, when this is also new territory (no prior daemon/IPC experience on this project)?

- **Decision: not now.** A daemon is not a one-time build — it's ongoing complexity (process lifecycle, crash recovery, keeping in-memory state consistent when a repo gets re-indexed out from under a live daemon, concurrent-request handling) layered on top of a feature set that's about to change significantly in V2 (episodic memory, OLORIN.md loading) and beyond. Building it now risks designing it around V1's shape and having to redesign it once V2 changes what actually needs to stay warm. The measured cost (~6-10s import tax per call) is real but bounded — nothing in V1 is unusable because of it — making it exactly the kind of cost worth deferring in favor of features that expand what Olorin can do. Revisit post-V2/V3, once the feature set has settled enough that a daemon's shape wouldn't need a redesign the moment something new is built on top of it.
- **Interview angle:** recognizing when *not* to build infrastructure is as much a signal of engineering judgment as recognizing when to build it — this decision is logged with the same rigor as every "yes, build this" decision in this project, not glossed over as "we didn't get to it."

Status

- ☒ `think_override` extension point wired end-to-end (base → Groq → Ollama → LLMClient), unused by anything yet, ready for a future heavy- task routing decision
 - ☒ `think:false` confirmed genuinely honored via direct diagnostic, not inferred
 - ☒ V1.5 fully closed: measured, decided, documented — daemon deliberately deferred, not built
 - ☐ Next: V2 features (episodic memory, OLORIN.md, journal automation, entity memory) — per Section 11's V2 tier
-

2026-07-11 — V2: episodic memory (write

path + query path), one real bug found and fixed

memory/history.py built — Section 10's conversations table, live

- Schema exactly per Section 10: `timestamp`, `repo_path`, `user_query`, `agent_response`, `tools_used` (JSON array), `backend_used`, `complexity_score`.
- `log_conversation()` / `get_recent()` / `search_history()`. Connection pattern (`sqlite3.connect(config.MEMORY_DB_PATH)`, schema created on connect) mirrors `memory/cache.py` — same file, same DB, different table, consistent with the single `memory.db` already in use.
- **Scope decision, stated up front:** `search_history()` is keyword-overlap search (shared-word count between the query and each row's stored text), not semantic/embedding search. A proper semantic history search needs its own vector-store design (second Chroma collection? reuse the code collection with a type filter?) — a bigger decision than this pass warrants. Keyword overlap is a real, useful first cut; revisit with embeddings only if keyword recall proves too weak in practice.

tools/history.py + tools/registry.py — search_history wired as a fourth tool

- Same shape as `tools/search.py`: `honest {"error": ..., "results": []}` when nothing's been logged for a repo yet, rather than a silent empty list.
- `repo_root` injected by `execute_tool()`, same as every other tool — never trusted from model-supplied arguments.
- System prompt (`core/agent.py`'s `build_system_prompt()`) updated to describe all four tools, including when to reach for `search_history` ("what did I figure out about X before?"-style questions).

providers/base.py + core/llm_client.py — complexity_score threaded onto ProviderResponse

- `ProviderResponse` gained `complexity_score: int | None`, set by `LLMClient._route()` in auto mode (both the Groq-success and local-fallback branches), immediately after the provider call returns — providers themselves have no visibility into routing, only into answering, so this can't be set inside `GroqProvider/OllamaProvider`.
- Left `None` for forced-provider calls (`--provider X, --force-local`) and cache hits, where the scorer never actually runs — an honest gap, not a guess.

core/agent.py — Agent.run() wired to log every completed run

- Accumulates `tools_used` (tool names, in call order, duplicates kept — same style as the Engineering Journal's own "search_codebase (4x)" session summaries) and tracks `last_response` across the loop.
- `_log_conversation()` called at **every** return point — clean finish, the defensive `unexpected_finish_reason` branch, and `max-steps-reached` — not just the happy path, so a run that goes wrong still leaves a record of what was asked and what happened.
- `backend_used` derivation handles the ollama/groq asymmetry correctly: both local personas report `provider="ollama"` (see `providers/ollama_provider.py`), so `response.model ("boromir" / "faramir")` is used for local; `response.provider ("groq")` is used for cloud, since Groq's model name isn't the label Section 10's schema wants.
- Logging wrapped in `try/except` — a logging failure must never break the actual answer the user is waiting on.

cli.py — added history command

- `python cli.py history [--path] [--limit] [--`

`search]` — thin read-only wrapper over `memory/history.py`, added purely so the write path could be verified live without a raw `sqlite3` shell.

Real bug found + fixed: post-retrieval complexity scorer never actually saw retrieved content

- **Symptom:** first live verification of episodic memory logged `complexity_score=0` on every row, including a run where `search_codebase` had already returned 5 real chunks by the time the final answer was produced.
- **Root cause:** `Agent.run()` called `self.llm.chat(...)` without ever passing `retrieved_chunks` — on any step, including step 2+ after tool results were already sitting in the message history. `score_complexity()` (`core/llm_client.py`) always scored against an empty list as a result. Section 7's headline design goal — "classify complexity AFTER seeing retrieved chunks, not before, since the retrieved context is the real signal" — had genuinely never been exercised in the live ask path; it had only ever been proven in `llm_client.py`'s standalone smoke test with hardcoded chunks. Pre-existing since V1, surfaced now because episodic memory logging made the always-zero score visible and permanent for the first time.
- **Fix:** added `Agent._accumulate_retrieved_chunks()`, called after every tool execution in the loop, appending `search_codebase` chunk content and `read_file` content into a running `retrieved_chunks` list that's threaded into every subsequent `llm.chat()` call in the same run. `list_files` and `search_history` results are deliberately excluded — filenames and past conversations aren't the code-context signal this scorer is meant to weigh, and including them would conflate two orthogonal concerns.
- **Verified live:** ask "How is Rust being used in project?" — step 1 (pre-retrieval) still correctly scored low; `search_codebase` returned 5 chunks; step 2's `llm.chat()` call scored `complexity_score=42`

`COMPLEXITY_THRESHOLD(15)`, and the router escalated to Groq for the first time based on

real retrieved-content volume rather than query wording alone. `python cli.py history` confirmed the logged row shows `backend=groq complexity=42``, not a guessed or stale number.

- **Side effect flagged, not a regression:** queries that pull back a lot of `search_codebase` content will now legitimately escalate to Groq more often than V1's routing numbers suggested, since the scorer is finally seeing real retrieval volume. Worth watching Groq usage against the free-tier budget (Section 5) going forward, not something to patch.

Verified live, end-to-end

- ask "how does the agent loop handle tool calls?" → logged correctly, `backend=boromir, tools=search_codebase`.
- `history` and `history --search "tool calls"` → both return the logged row correctly.
- ask "have I asked about the agent loop before?" → agent chose `search_history` over `search_codebase` **on its own**, retrieved the actual prior conversation, and gave a grounded answer that correctly references the earlier question rather than hallucinating a summary — the real test of whether the tool is useful, not just wired up.
- ask "How is Rust being used in project?" → confirmed the `retrieved_chunks` fix live (see above), correctly escalated to Groq, logged `complexity=42`.

Status

- ☒ `memory/history.py` — episodic memory write path (Section 10), verified live
- ☒ `search_history` query path — keyword-overlap search, wired as a fourth agent tool, verified live (agent chose it correctly on a memory-shaped question)
- ☒ `complexity_score` threaded through `ProviderResponse` → `LLMClient._route()` → episodic memory log

- ☒ `cli.py history` command for live verification/browsing
 - ☒ Real bug found and fixed: post-retrieval complexity scorer never received real `retrieved_chunks` in the live agent path — fixed via `_accumulate_retrieved_chunks()`, verified live with a real Groq escalation driven by real chunk volume
 - ☐ Next: remaining V2 features (OLORIN.md per-repo config loading, Engineering Journal automation, entity memory) — per Section 11's V2 tier
-

2026-07-11 (continued) — Knowledge Capsules, pulled forward from V3

Design decision: entity memory deferred, Knowledge Capsules built instead

- Discussed entity memory's design gap directly (Section 10's `entities` table was a stub with no resolved population mechanism, type scope, or update policy — unlike episodic memory, which had a fully specified schema before any code was written) versus Knowledge Capsules, listed under V3 but with a clearer value proposition and less open design risk.
- **Decision: build Capsules now, defer entity memory.** Same judgment as the daemon deferral — recognizing a lower-value/higher-uncertainty item and choosing the better-specified, higher-payoff one out of strict roadmap order, logged with the same rigor as any other decision rather than silently reordered.

Design conversation: context budget

- Boromir/Faramir run at 16K ctx (Modelfile-set, independent of the underlying models' larger native max context). The system prompt is resent in full on every step of the ReAct loop (no persistent server-side context between calls), and `read_file` alone can return up to ~5,000 tokens in a single call — so the real risk wasn't

repo size, it was capsule text competing with growing tool-result content for the same fixed budget, worst exactly on the multi-step queries where the budget is already tightest.

- **Resolved design:** inject capsules into the system prompt for step 1 only (when the model is deciding whether/what to call, which is the only point capsules actually help with), then rebuild `messages[0]` without them before step 2 — capsules are dead weight once the model's reasoning over concrete retrieved content instead of trying to orient itself. Paired with a hard length cap enforced in the generation prompt itself ("3-4 sentences, under 100 words") so the step-1 tax is small and predictable regardless of module size.
- **Deliberately deferred:** per-query relevance filtering (only inject capsules for modules that seem related to the question) — the actual answer for large repos, but solving a problem this repo's scale (5-6 modules) doesn't have yet. Same discipline as the daemon deferral: revisit once a repo is big enough that even step-1-only, length-capped capsules start crowding the budget.

memory/capsules.py + indexer/capsules.py — store/generate split

- Storage (`memory/capsules.py`): `capsules(repo_path, module, summary, source_files, last_updated)`, `upsert-by-(repo_path, module)`, `needs_regeneration()` comparing the current (path, hash) set per module against what the stored capsule was generated against — same hash-comparison philosophy as `cli.py`'s per-file reindex-skip logic, applied at module granularity.
- Generation (`indexer/capsules.py`): takes an LLM client as a plain duck-typed argument rather than importing `core.llm_client` directly — `indexer/` has had no dependency on `core/` up to this point, and there was no reason to introduce one; `cli.py`, which already orchestrates across both, wires them together.
- **Faramir generates capsules, not Boromir** — Section 2's lore assigns Faramir to general reasoning/synthesis and Boromir to fast task-focused tool execution; summarizing "what does this

module do" is squarely the former. The known Faramir risk logged elsewhere (<think> blocks confusing tool-call parsing) doesn't apply here — capsule generation makes no tool calls, it's a single plain completion.

- Module grouping uses a dedicated extension allowlist in `cli.py` (`_CODE_EXTENSIONS`) rather than trusting the walker's language field — keeps capsule generation decoupled from `chunker.py`'s internal language-support matrix, which this feature has no need to know about. Loose root-level files (`config.py`, `cli.py`) fall under a synthetic "root" module.

`cli.py` index — capsule generation wired in after the existing pipeline

- Runs after the walk/chunk/embed/store loop completes: groups files by module, checks `needs_regeneration()` per module, regenerates via Faramir only for modules whose file-hash set actually changed.
- Wrapped in try/except per module — a capsule generation failure (Ollama down, model error) must not lose the indexing work that already succeeded above it.

`core/agent.py` — step-1-only injection implemented

- `build_system_prompt(repo_root, include_capsules=True)` — capsule section built from `memory.capsules.get_all_capsules()`, wrapped in try/except (a capsule read failure shouldn't block answering).
- `Agent.run():` initial `messages[0]` includes capsules; immediately after step 0's tool-call handling, `messages[0]` is unconditionally rebuilt with `include_capsules=False` before the loop continues to step 2. Guarded by `if step == 0` so this only fires once per run.

Verified live, end-to-end

- `python cli.py index .: 30 files walked, 16 (re)indexed, 7`

capsules generated (tools, memory, core, providers, root, indexer, indexer_core) at 12-26s each (Faramir `think:true` — deliberation is the point, cost is acceptable since this runs at index time, off the interactive critical path).

- Re-running `index` immediately after: Capsules: 0 generated, 7 unchanged — confirmed the hash-based skip logic works, not just the generation path.
- ask "what is this project and how is it structured?" — answered in a **single step, zero tool calls**, correctly synthesizing all seven modules' roles from capsule text alone (correctly distinguished `memory/capsules.py` storage from `indexer/capsules.py` generation — real synthesized understanding, not a lucky guess). This is the feature's actual payoff: an orientation-style question resolved without a single exploratory `list_files/read_file` round-trip.
- Re-ran the same style of query warm (Boromir already loaded): 13.3s total, confirming the earlier 40s `ask` was Ollama's cold-start model-swap cost (already measured at ~41s back on 2026-07-06), not capsule injection overhead — capsules are not meaningfully adding to per-call latency.
- ask "what does the `score_complexity` function do, read the actual code" — forced a real multi-step run: step 1 (capsules present) → `search_codebase` → step 2 (capsules correctly stripped from `messages[0]`, no malformed history, no crash) → `read_file` → step 3 → final answer from Groq, accurate and grounded in the real code. This is the step-1-only stripping logic actually exercised under real multi-step load for the first time, and it held up cleanly. Also reconfirmed the `retrieved_chunks` fix from earlier today (real Groq escalation, driven by real chunk volume) still works correctly alongside capsule injection.

Status

- ☒ `memory/capsules.py` + `indexer/capsules.py` — store/generate split, verified live
- ☒ `cli.py index` — capsule generation + hash-based skip, verified live (both the generate path and the skip path)
- ☒ `core/agent.py` — step-1-only capsule injection, verified live

- under both a single-step and a real multi-step run
 - ☒ Zero new bugs found this session — first V2 feature session without one, worth noting for contrast with episodic memory's session
 - ☐ Next: remaining V2 features (OLORIN.md per-repo config loading, Engineering Journal automation, entity memory) — per Section 11's V2 tier
-

2026-07-11 (continued) — OLORIN.md loading, a real list_files bug, and index-on-demand

OLORIN.md per-repo config loading — shipped

- `core/agent.py`'s new `load_olorin_md()`: reads `OLORIN.md` from the repo root once per `Agent` instance, honest `None` when absent, capped at 4,000 chars defensively.
- **Deliberately never stripped**, unlike Knowledge Capsules — this is explicit user-authored instruction ("never suggest SQLAlchemy"), not orientation help the model can safely stop needing once it's grounded in retrieved code. Threaded through both the step-1 and step-2+ `build_system_prompt()` calls in `Agent.run()`.
- Wrote a real `OLORIN.md` for this repo (project conventions, persona roles, where provider quirks belong) as both the working example the doc always promised and the live test fixture.
- **Verified live, two ways**: a query about refactoring the agent loop correctly cited the "hand-written ReAct, not LangChain" rule from `OLORIN.md` on step 1; a forced multi-step query (`read_file` → Groq) correctly cited the "provider quirks belong in `ollama_provider.py`" rule on its **step 2 answer**, confirming persistence past the point where capsules get stripped — the actual behavioral difference this design was built for, not just presence-of-a-section.

Real bug found + fixed: `list_files` had no result-size cap (bug #9)

- **Symptom, found while verifying OLORIN.md on a real repo (Vorticity):** ask "what is this project?" on an unindexed repo took 115 seconds on its final step. Root cause wasn't OLORIN.md — it was `list_files({'pattern': '*'})` returning an entirely unbounded file list (Next.js build output, ML checkpoints, notebook checkpoints, none of it excluded) that the local model then had to process in full before answering.
- **Notable interaction with the complexity scorer:** `list_files` results are deliberately excluded from `Agent._accumulate_retrieved_chunks()` (filenames aren't code-context signal, see bug #8's fix) — which means an oversized `list_files` result never had a chance to correctly escalate to Groq. It just silently cost real local latency instead, invisible to the one system designed to catch "this query got expensive."
- **Fix:** `tools/reader.py`'s `list_files()` gained a `max_files=200` hard cap (mirroring `read_file`'s existing `max_chars` pattern) plus `total_matched/truncated` fields so the agent knows when to narrow its pattern instead of assuming it saw everything. Expanded `IGNORED_DIRS` to include `.next`, `dist`, `build`, `target`, `.ipynb_checkpoints`, `.pytest_cache`, `.mypy_cache` — the actual bulk of the 115s repo.
- **Verified live:** re-running the original repo's `list_files` call came back in ~15s total (both steps) instead of the original single 115s step; a targeted `*.py` pattern query on the same repo returned a clean 9-file list in under 15s combined.

Design conversation: indexing shouldn't be a step the user has to remember

- Raised directly: for a product Olorin is meant to become, requiring a separate manual `index` before every new repo's first ask is exactly the kind of implementation detail ("think in terms of

embeddings and vector databases") a real user shouldn't have to know exists. Users think in terms of projects and questions.

- **Resolved design, confirmed directly:** auto-indexing should NOT mean "reindex everything on every call" — the hash-based skip logic already built (Section 8) means checking freshness on every `ask` is cheap when nothing changed (a fast walk + hash comparison, no re-chunk/re-embed/ re-capsule work), and only pays real cost when something's actually different. This reframes indexing from an explicit user-owned step to an automatic, lazy-but-always-checked precondition of every query — sometimes called index-on-demand or self-indexing RAG. The retrieval architecture itself (chunk → embed → vector search) doesn't change; only who owns freshness does.
- **Honest, not hidden, tradeoff:** because the daemon was deliberately deferred (V1.5), there's no background process to push first-time/ changed-file indexing work off the interactive path — it has to happen inline, blocking, before the agent starts reasoning. This is a real cost the design doesn't eliminate, just makes automatic instead of manual. Addressed with explicit messaging rather than a silent hang: a "first time indexing—this may take a minute" message when the collection is empty, and a one-line "repo changed since last index" summary when real work happened on a repeat call — silence on the common case (nothing changed).

`cli.py` refactored: `_index_repo()` extracted, wired into `ask`

- The full walk/hash-skip/chunk/embed/store/capsule pipeline, previously inline in the `index` command, is now `_index_repo(repo_path, verbose=True|False).index` is now a thin wrapper (`verbose=True`, unchanged behavior/output). `ask` calls it (`verbose=False`) as an automatic precondition before constructing the `Agent`, checking `collection.count() == 0` beforehand to decide whether to print the first-time message.
- Added `--skip-index` as an explicit escape hatch (advanced/debugging use), consistent with the existing `--provider/--`

`force-local` pattern of giving explicit overrides rather than only ever inferring behavior.

- **Verified live, all three states on a real never-indexed repo (tutorial-video-generator):**

1. First `ask` — printed the first-time message, indexed + generated 7 capsules silently (no file-by-file spam), answered accurately in a single step from capsule text alone.
2. Second `ask`, nothing changed — completely silent on indexing, single-step answer, notably *more* technically specific (correctly named WhisperModel, edge TTS, FFmpeg, headless Chromium) purely from capsule text — real implementation detail surviving the capsule length cap, not just generic descriptions.
3. `--skip-index` — correctly skipped the freshness check with no message either way, while still correctly using the capsules already generated in step 1 (capsule injection reads storage independent of whether the freshness check ran).

Status

- ☒ OLORIN.md per-repo config loading — shipped, verified live (both presence-of-rule and persistence-past-step-1)
 - ☒ Real bug found and fixed: `list_files` had no result-size cap (bug #9), invisible to the complexity scorer by design — fixed with a hard cap + expanded noise-directory list, verified live
 - ☒ Indexing reframed from a manual step to an automatic, hash-cheap precondition of `ask` ("index-on-demand") — `_index_repo()` extracted and shared between `index` and `ask`, verified live across first-time/unchanged/skip-index states on a real never-indexed repo
 - ☐ Next: remaining V2 features (Engineering Journal automation, entity memory once designed) — per Section 11's V2 tier
-

2026-07-12 — Direct-address routing, three

real bugs it surfaced, one known limitation logged

Direct-address routing override — shipped

- Raised directly: routing purely by the complexity scorer means there's no way to force a specific backend through phrasing alone, since the scorer is deliberately driven by retrieved-content volume, not query wording (Section 7). Wanted an explicit override expressed inline ("Groq, ...", "Boromir, ...") rather than only via the `--provider` flag.
- **Explicitly distinguished from a rejected idea:** Section 13 already rejected keyword-based *difficulty* classification. This is not that — it's a deterministic, narrow address parser functionally identical to the `--provider` flag, just inline. Nothing about routing is inferred; the user is stating a choice.
- `core/agent.py`'s new `parse_direct_address()`: matches only at the exact start of a query ("Boromir, ...", "Groq: ...", "Olorin ..."), maps onto the same `force_provider` strings the CLI already uses ("olorin"→"groq" per Section 2's lore — Olorin IS the Groq persona). Only applies when the caller didn't already force a provider — an explicit `--provider/--force-local` flag always wins. Strips the address from what reaches the model; keeps the original raw text (`original_query`) for episodic memory logging, since the user really did type it and it should stay searchable as what actually happened.
- **Verified live:** correct routing and stripping confirmed in logs for all three names; precedence (`--provider` beats an in-text address) confirmed separately; episodic memory confirmed to retain the original phrasing, not the cleaned version.

Real bug found + fixed: forced-provider calls had no fallback (bug #10)

- **Symptom:** the very first live test of direct-address routing crashed

the CLI outright — ask "Groq, what is this project?" hit a real Groq 413 ("Request too large", 39,218 tokens against a 12,000 TPM free-tier limit) and the exception propagated all the way up as a raw traceback. Auto-mode's Groq attempt has always been wrapped in `try/except ProviderError` with a fallback to local; the forced-groq and forced-local branches in `LLMClient._route()` never were. This gap predates direct-address routing (the `--provider groq` flag had the same hole) — it just went unnoticed because forcing Groq explicitly was rare until this session made it a natural, frequent thing to type.

- **Fix:** forced-groq now wraps the call in the same `try/except ProviderError` pattern as auto-mode, falling back to `self.default_local` (boromir) on any failure, tripping the circuit breaker if it was a rate limit. `cli.py`'s `ask` also gained a top-level `try/except` around `agent.run()` as a safety net for whatever a forced-local failure (e.g. Ollama not running) has no further automatic fallback for without contradicting the user's explicit choice — a clean red error message instead of a raw traceback either way.
- **Verified live under a genuinely new failure mode**, not just the original trigger: a later run hit a *different* Groq error entirely (a malformed tool-call, 400 `tool_use_failed`, from `llama-3.3-70b-versatile` itself) and the fallback caught it exactly the same way — good evidence the fix is generically robust, not narrowly patched to one symptom.

Real bug found + fixed: chunks had no size ceiling (bug #11)

- **Root cause:** `indexer/chunker.py`'s whole-file fallback (used for any language without a tree-sitter grammar — markdown, notably) has never had a size cap; the chunker's own docstring flagged this and explicitly deferred it to the embedder layer. Never implemented until now. `OLORIN_PROJECT.md` and `ENGINEERING_JOURNAL.md`, grown very large from this session's own documentation work, were the concrete trigger for bug #10's 413 — a single whole-file chunk large enough to blow a request's token budget on its own.

- **Fix:** `indexer/embedder.py`'s `_split_oversized_chunks()`, called inside `embed_chunks()`, splits any chunk over `MAX_CHUNK_CHARS` (2000, roughly bge-small's own ~512-token limit — keeps what's embedded and what's stored/returned honestly in sync) into sequential sub-chunks. Deliberately a blunt line-budget split, not markdown/AST-aware — a generic fix that protects against any oversized chunk, not just markdown; logged explicitly as a known simplification, not a final answer (see the known limitation below).

Real bug found + fixed: the chunk-split fix's own bug — duplicate IDs (bug #12)

- **Symptom:** the very next live test of bug #11's fix crashed indexing entirely — `ChromaDB DuplicateIDError`. `store.py`'s `_chunk_id()` hashes `file + lines`; the first version of `_split_oversized_chunks()` numbered each sub-chunk's lines relative to the `CHUNK`'s own extracted text (always starting at 1), not the file's true absolute line numbers. Any two oversized chunks in the same file with roughly similar size would deterministically produce identical relative ranges ("1-55", "56-100", ...) — not a rare coincidence, a guaranteed collision for any file with 2+ chunks needing splitting.
- **Fix:** anchored each sub-chunk's line numbers to the original chunk's true starting line (parsed from its own `lines` metadata), so ranges reflect real file positions and stay genuinely unique. Also added a defense-in-depth guard in `store.py`'s `upsert_chunks()` — de-duplicates by ID before calling Chroma's `upsert()`, which otherwise aborts an ENTIRE file's upsert (every legitimate chunk in the batch, not just the colliding one) on a single collision. A backstop for any future collision cause, not a substitute for fixing root causes at the layer that produces them.
- **Verified live:** the guard fired correctly (`WARNING: 1 duplicate chunk id(s) collapsed before upsert`), indexing completed without crashing, and the subsequent `ask` call succeeded end-to-end.

Known limitation logged, not fixed: retrieval miss on a real question

- **Symptom:** ask "Boromir, summarize what OLORIN_PROJECT.md says about the daemon deferral" answered that the document "does not directly discuss" the daemon deferral — false; Section 11's entire V1.5 tier and a dedicated Section 13 entry cover it extensively.
- **Assessment:** likely retrieval missing the right chunk after bug #11's line-budget split, not the model fabricating — notably it did NOT hallucinate a wrong daemon-deferral narrative, it honestly reported what it could and couldn't find, which is the safer failure mode. Plausibly exactly the caveat already logged in `_split_oversized_chunks()`'s own docstring: a blunt line-budget split can cut through a topical section instead of respecting document structure, hurting retrieval precision on long docs specifically.
- **Decision: logged as a known limitation, not investigated further this session.** A markdown/header-aware split would be the real fix if this proves to matter in practice — deliberately not chased now, consistent with the project's standing discipline of not building past what's actually been measured to be a problem.

Status

- ☑ Direct-address routing — shipped, verified live including precedence and episodic-memory-preserves-original-text behavior
- ☑ Bug #10 (forced-provider no fallback) — fixed, verified live under two genuinely different Groq failure modes
- ☑ Bug #11 (chunks had no size ceiling) — fixed
- ☑ Bug #12 (chunk-split fix's own duplicate-ID bug) — fixed, with a defense-in-depth guard added alongside the root-cause fix
- ☑ Known limitation logged: retrieval precision on long docs after line-budget splitting — deliberately deferred, not fixed
- ☐ Next: Engineering Journal automation, Together/Cerebras providers (per user direction, entity memory pushed further out — finishing- stage feature, not V2)

(continued in `ENGINEERING_JOURNAL.md`)